Informe Proyecto 1

Departamento de Ingeniería de Sistemas y Computación

ISIS 1511 - Fundamentos de Bases de Datos

Juan David Vasquez Leyva - 202426062

Brayan Stiven Bernal Marin - 202514036

Joshua Barajas Holguin - 202511642

Universidad de los Andes

ENTREGA 1 **3**
1. El Problema que se Resuelve **3**
2. Análisis **3**
3. Reglas del negocio (Relaciones) **4**
4. Modelo Conceptual E/R **4**
5. Transformación a Modelo Relacional **6**
6. Normalización **7**
6.4 Forma Normal De Boyce-Codd (FNBC) **9**
7. Restricciones **9**
ENTREGA 2 **10**
1. Análisis y revisión del modelo de datos 10
2. Implementación de la Base de Datos 10
3. Población de las tablas 10
4. Requerimientos Funcionales 11
5. Requerimientos Funcionales de Consulta 14
6. DESARROLLO DE LA APLICACIÓN EN APEX 16
7. ESCENARIOS DE PRUEBA 16

ENTREGA 1

1. El Problema que se Resuelve

La cadena hotelera Dann-Alpes opera múltiples hoteles distribuidos en distintas ciudades de Colombia y requiere un sistema de información que permita gestionar de manera eficiente y centralizada sus operaciones. Actualmente, la gestión manual de reservas, habitaciones y servicios puede generar inconsistencias en la información, dificultades en el control de disponibilidad y errores en el cálculo de precios.

El proyecto tiene como objetivo diseñar y modelar una base de datos que permita solucionar los siguientes problemas principales:.

- Gestión centralizada.
- Registro y consulta de disponibilidad por fecha.
- Administración de reservas realizadas por clientes.
- Cálculo adecuado de precios según temporada y servicios.
- Soporte para consultas.

La solución propuesta se desarrolla a partir del diseño conceptual y relacional de la base de datos mediante Oracle Developer Data Modeler, herramienta con la cual se modelan correctamente las entidades del negocio, sus relaciones, restricciones y atributos correspondientes.

2. Análisis

A partir del caso de estudio se identificaron los siguientes elementos:

- La cadena opera en múltiples ciudades.
- Cada ciudad puede tener varios hoteles.
- Cada hotel ofrece distintos tipos de habitación.
- La disponibilidad de habitaciones depende del hotel.
- Los servicios adicionales pueden variar según el tipo de habitación de cada hotel.
- Los clientes deben registrarse para realizar reservas.
- Una reserva puede incluir distintos servicios adicionales.
- Existen dos tipos de ***usuario***:
    1. **Cliente**: Interactúa con el sistema para realizar y gestionar reservas.
    2. **Admin**: Es el responsable de configurar y mantener la información del sistema.

Además, se decidió modelar la disponibilidad mediante una entidad intermedia entre hotel y Tipo de habitación, permitiendo manejar configuraciones específicas por cada hotel.

3. Reglas del negocio (Relaciones)

3.1 Reglas sobre Hoteles y ciudades

- Una ciudad puede tener uno o más hoteles de la cadena
- Un hotel pertenece a exactamente una ciudad.
- Cada hotel puede ofrecer múltiples tipos de habitación con cantidades específicas disponibles.
- Un tipo de habitación puede estar disponible en múltiples hoteles.

3.2 Reglas sobre Habitaciones

- Cada habitación física pertenece a un hotel y a un tipo.
- Un tipo de habitación define el costo de base en temporada alta o baja, dimensiones, tipo de vista y capacidad.
- Cada tipo de habitación puede tener uno o más tipos de camas.
- Las comodidades se definen por tipo de habitación, no por habitación individual.

3.3 Reglas sobre Servicios

- Los servicios adicionales se habilitan por tipo de habitación en cada hotel.
- Un servicio puede ser de costo fijo o por consumo.
- Un servicio se puede aplicar por reserva.
- La cancelación flexible es un servicio de costo fijo que permite cancelar hasta 3 días antes.

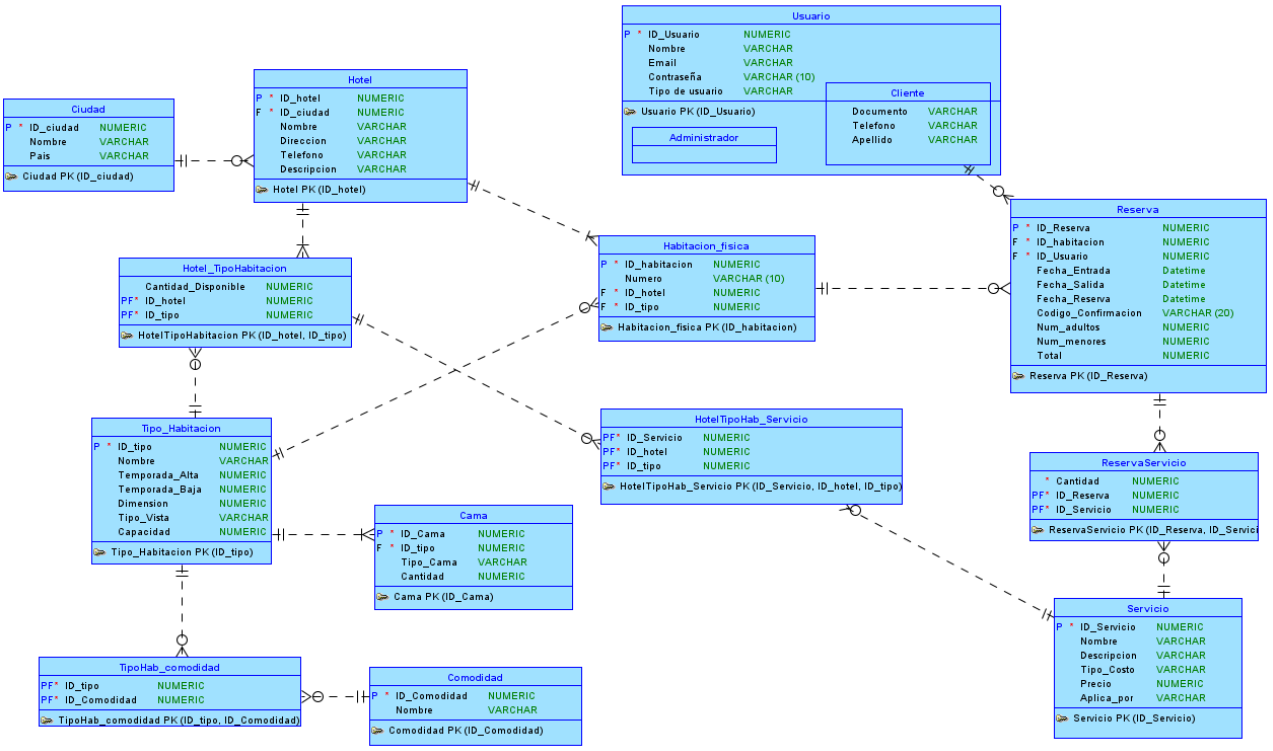
3.4 Reglas sobre Reservas

- Una reserva está asociada a un cliente y una habitación física.
- La fecha de salida debe ser estrictamente después de la fecha de entrada.
- Solo se puede reservar si hay disponibilidad del tipo de habitación en el hotel.
- El total de la reserva se calcula según la temporada, número de noches y servicios.
- Una reserva puede incluir 0 o más servicios adicionales.
- Un cliente puede tener 0 o muchas reservas.

4. Modelo Conceptual E/R

4.1 Entidades

Primero se identificaron las entidades que componen el sistema, junto con los atributos necesarios para describir cada una. Estas entidades permiten representar adecuadamente la información del negocio y establecer las relaciones entre los distintos componentes del sistema.



En la siguiente tabla se muestra un resumen donde se aprecian mejor las entidades y atributos importantes:

Entidad	Atributos principales
Ciudad	ID_Ciudad, Nombre, País
Usuario	ID_Usuario, Nombre, Email, Contraseña, Tipo
Hotel	ID_Hotel, ID_Ciudad (FK), Nombre, Dirección, Teléfono, Descripción
Tipo_Habitacion	ID_Tipo, Nombre, Temporada_Alta, Temporada_Baja, Dimensión, Tipo_Vista, Capacidad
Habitacion_Fisica	ID_Habitacion, ID_Hotel (FK), ID_Tipo (FK), Número
Cama	ID_Cama, ID_Tipo (FK), Tipo_Cama, Cantidad
Comodidad	ID_Comodidad, Nombre
Servicio	ID_Servicio, Nombre, Descripción, Tipo_Costo, Aplica_Por, Precio
Reserva	ID_Reserva, ID_Usuario (FK), ID_Habitacion (FK), Fecha_Entrada, Fecha_Salida, Fecha_Reserva, Codigo_Confirmacion, Num_Adultos, Num_Menores, Total

4.2 Especialización

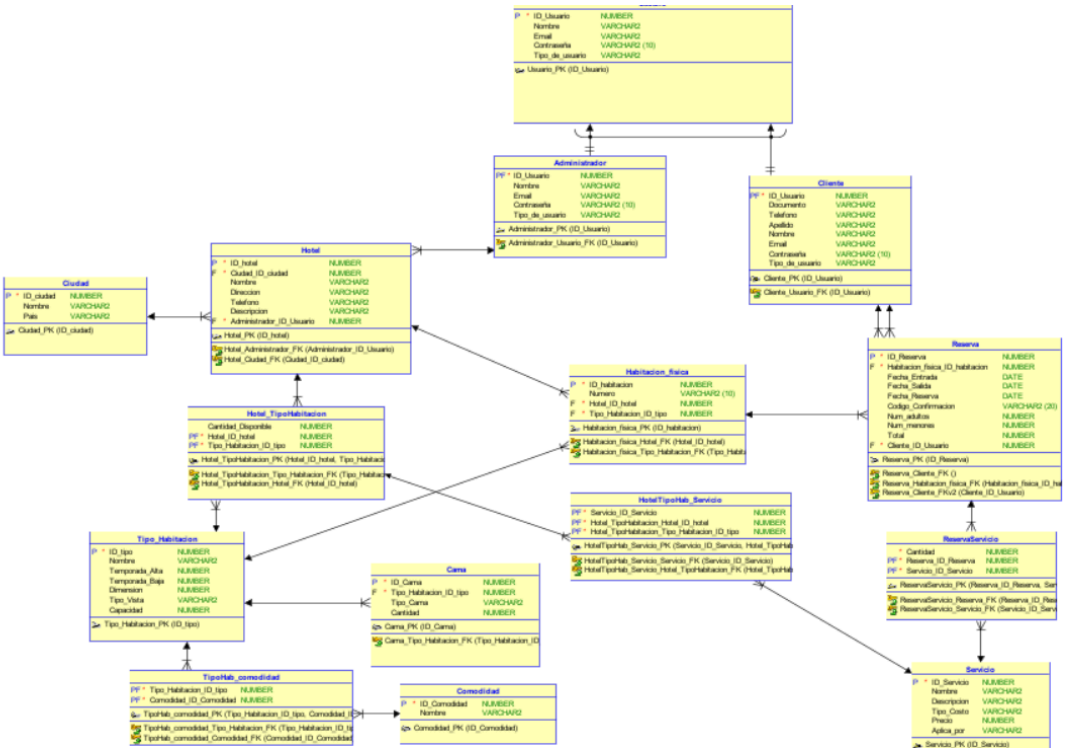
La entidad Usuario se especializa en dos sub-entidades mediante una relación de herencia:

1. ***Cliente:*** Subentidad de Usuario que agrega los atributos Documento, Teléfono y Apellido. Se distingue por el valor cliente en el atributo Tipo.

2. ***Admin:*** subentidad de Usuario sin atributos propios adicionales. Hereda todos los atributos de Usuario. Se distingue por el valor 'admin' en el atributo Tipo.

5. Transformación a Modelo Relacional

Una vez definido el modelo conceptual con sus respectivas entidades y relaciones, se realizó el proceso de ingeniería para transformarlo en el modelo relacional.



Cada entidad fuerte con existencia propia se convierte directamente en tabla, conservando el Pk que fue definido en el modelo lógico.

- Usuario -> Tabla *Usuario (PK: ID_Usuario)*
- Ciudad -> Tabla *Ciudad (PK: ID_Ciudad)*
- Hotel -> Tabla *Hotel (PK: ID_Hotel)*
- Tipo_Habitación -> Tabla *Tipo_Habitación (PK: ID_tipo)*
- Habitación_Física -> Tabla *Habitación_Física (PK: ID_Habitación)*
- Cama -> Tabla *Cama (PK: ID_Cama)*
- Comodidad -> Tabla *Comodidad (PK: ID_Comodidad)*
- Servicio -> Tabla *Servicio (PK: ID_Servicio)*
- Reserva -> Tabla *Reserva (PK: ID_Reserva)*

5.2 Relaciones 1:N

Las relaciones uno a muchos no generan una tabla nueva. La FK se coloca al lado N apuntando al lado 1:

- Ciudad (1) -> Hotel (N) : Se agrega *ID_Ciudad* como FK en la tabla Hotel
- Hotel (1) -> Habitación_Física (N): Se agrega *ID_Hotel* como FK de Habitación_Física
- Tipo_Habitación (1) -> Habitación_Física (N) : Se agrega *ID_Tipo* como FK en Habitación_Física
- Tipo_Habitación (1) -> Cama (N) : Se agrega *ID_Tipo* como FK en Cama
- Cliente (1) -> Reserva (N) : Se agrega *ID_Usuario* como FK en Reserva
- Habitación_Física (1) -> Reserva (N) : Se agrega *ID_Habitación* como FK en Reserva

5.3 Relaciones N:M

En este caso hicimos que las relaciones muchos a muchos generaran una tabla intermedia con PK compuesta formada por las PKs de las entidades relacionadas:

- Hotel N:M Tipo_Habitacion -> Tabla Hotel_TipoHabitacion (PK: ID_Hotel + ID_Tipo). Incluye un atributo propio: Cantidad_Disponible.
- Tipo_Habitación N:M Comodidad -> Tabla TipoHab_Comodidad (PK: ID_Tipo + ID_Comodidad). Sin atributos propios.
- Reserva N:M Servicio -> Tabla ReservaServicio (PK: ID_Reserva + ID_Servicio). Incluye atributo propio: Cantidad.

5.4 Especializaciones (Subentidades)

La herencia Usuario -> Admin / Cliente se implementa usando la PK de la superentidad a las subentidades como PK + FK:

- Admin: Tabla con ID_Usuario como PK y FK apuntando a Usuario. Sin atributos propios.
- Cliente: Tabla con ID_Usuario como PK y FK apuntando a Usuario. Agrega: Documento, Teléfono y Apellido.

6. Normalización

Se verificó el nivel de normalización de cada relación del modelo. El análisis muestra que todas las tablas cumplen con la Formal Normal de Boyce-Codd (FNBC) desde el diseño inicial.

6.1 Primera Forma Normal (1FN)

Una relación está en 1FN si todos sus atributos son atómicos.
Todas las tablas del modelo cumplen 1FN porque:

- Ningún atributo contiene listas o conjuntos de valores. En el caso de las comodidades se separaron en la tabla Comodidad y TipoHab_Comodidad.
- Los tipos de cama se separaron en la tabla Cama en lugar de almacenarse como un campo con múltiples valores en TIpo_Habitacion.
- Los servicios adicionales de cada reserva se manejan en ReservaServicio, no como un campo de texto.
- Cada columna tiene solo un único valor por fila en todas las tablas.

6.2 Segunda Forma Normal 2FN

Una relación está en 2FN si está en 1FN y todos los atributos no clave depende de la PK completa. Aplica especialmente para tablas con PK compuesta.

Verificación de tablas con PK compuesta:

- Hotel_TipoHabitacion (PK: ID_Hotel + ID_Tipo): El único atributo no clave es Cantidad_Disponible, que dependen de ambas llaves juntas.
- TipoHab_Comodidad (PK: ID_Tipo + ID_Comodidad): No tiene atributos propios. Cumple 2FN.
- HotelTipoHab_Servicio (PK: ID_Hotel + ID_Tipo + ID_Servicio): No tiene atributos propios. Cumple con 2FN.
- ReservaServicio (PK: ID_Reserva + ID_Servicio): El atributo Cantidad depende de ambas llaves juntas

Las demás tablas tienen PK simple, por lo que 2FN se cumple automáticamente.

6.3 Tercera Forma Normal 3FN

Una relación está en 3FN si está en 2FN y no existen dependencias transitivas entre los atributos.

Ninguna tabla presenta dependencias transitivas porque:

- En **Usuarios**: Nombre, Email, Contraseñas y Tipo dependen directamente de ID_Usuario, no entre sí.
- Un **Hotel**: Todos los atributos dependen de ID_Hotel, ID_Ciudad es FK, no genera transitividad porque Nombre y Dirección no dependen de ID_Ciudad sino de ID_Hotel.
- En **Tipo_Habitación**: Temporada_alta, Temporada_baja, Dimension, Tipo_Vista y Capacidad dependen directamente de ID_Tipo.
- En **Reserva**: Total depende de ID_Reserva. No hay atributos que dependen de otros atributos no clave.

6.4 Forma Normal De Boyce-Codd (FNBC)

Para toda dependencia funcional X -> Y,X es superllave.

En todas las tablas las llaves primarias determinan todos los atributos y no existen determinantes que no sean supera llaves.

Por lo tanto, el modelo se encuentra en FNBC.

7. Restricciones

Las siguientes restricciones fueron implementadas en el modelo:

- PK en todas las entidades.
- FK con integridad referencial. Esto quiere decir que no puede existir un registro que haga referencia a otro que no existe.
- NOT NULL en llaves primarias.
- CHECK (Num_adultos >= 0)
- CHECK (Num_menores >= 0)
- CHECK (Fecha_salida > Fecha_entrada).
- UNIQUE (Correo en Usuario)

ENTREGA 2

1. Análisis y revisión del modelo de datos

Cambios realizados en el Modelo E/R para esta entrega

Para esta entrega decidimos adaptar nuestro modelo a uno igual al que se nos envió como ejemplo como referencia así que contamos con las mismas tablas y descartamos algunas que no eran necesarias ya que en la retroalimentación de la primera entrega se nos mencionó que no eran tantas, entonces contamos con: Ciudades, Usuarios, Hoteles, TiposHabitaciones, Habitaciones, Reservas y Servicios.

2. Implementación de la Base de Datos

Se ocuparon los siguientes estándares:

- Uso de nombres en mayúsculas para tablas y atributos para mantener consistencia
- Definición de llaves primarias
- Definición de llaves foráneas
- Aplicación de restricciones CHECK para limitar valores válidos
- Definición de campos NOT NULL en atributos obligatorios

Tablas creadas:

-Ciudades
-Usuarios
-Hoteles
-TiposHabitaciones
-Habitaciones
-Reservas
-Servicios

3. Población de las tablas

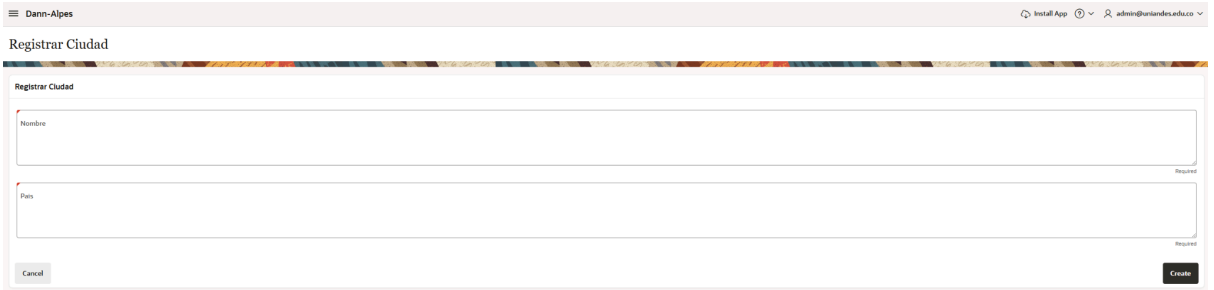
- Para poblar la base de datos nos apoyamos de **IAG** para que nos ayudará llenando con datos genéricos cada una de las tablas mencionadas en el punto 1
- Se insertó lo que se solicitaba:
  - 15 hoteles diferentes
  - 300 usuarios
  - Generar aprox 300 reservas haciendo una media de 20 clientes por hotel
  - Crear habitaciones y tipos de habitaciones asociadas a cada hotel
  - Asignar servicios a reservas y habitaciones

4. Requerimientos Funcionales

Los requerimientos fueron implementados en Apex como páginas de tipo formulario a las cuales el Admin puede acceder para hacer los cambios y agregar datos a la base de datos

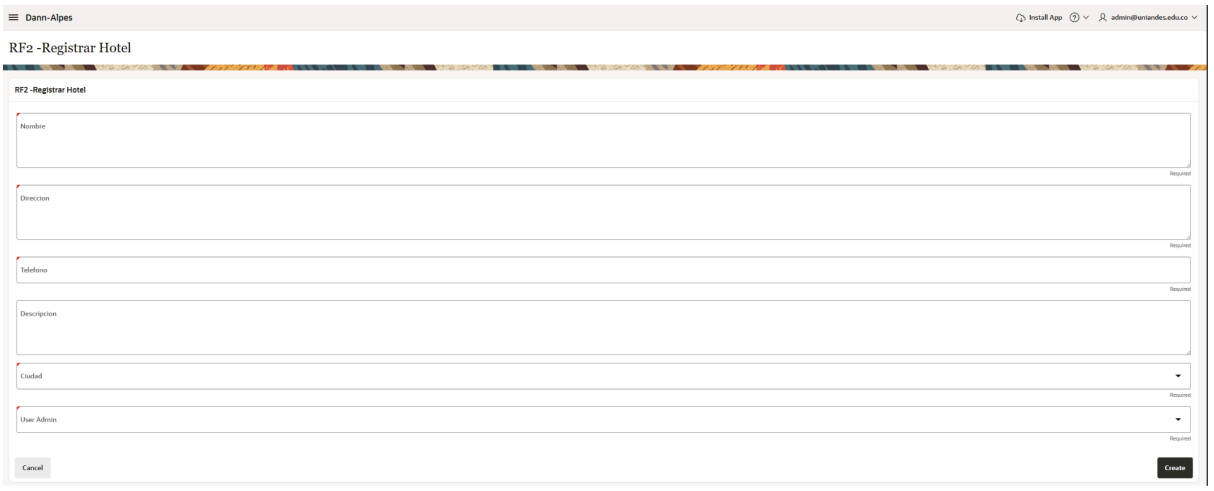
Los requerimientos elaborados fueron:

-RF 1 - Registrar una Ciudad



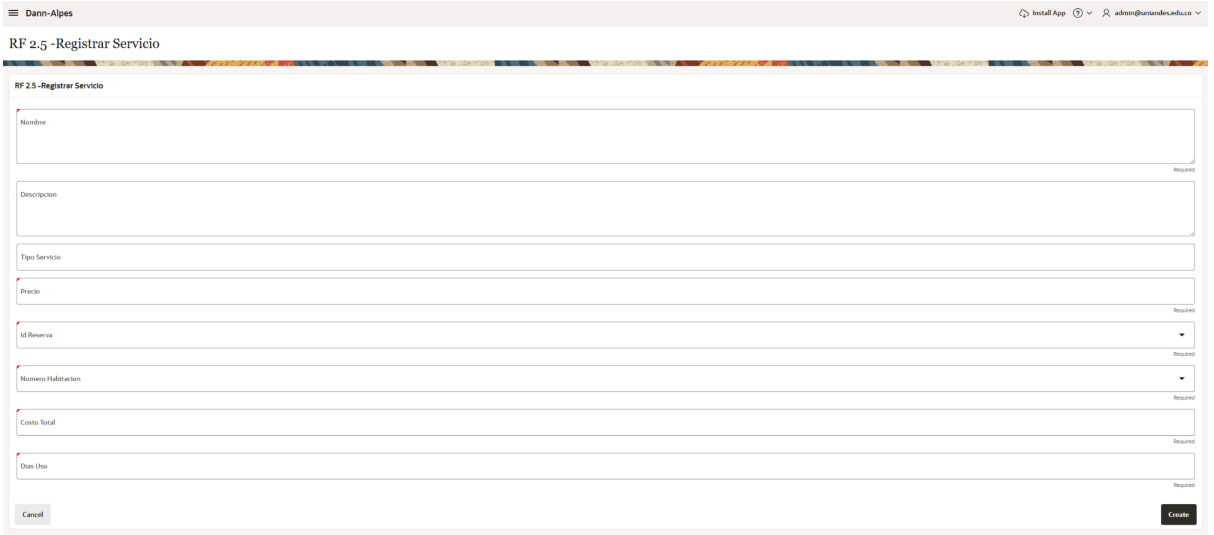
Solicita Nombre y País al Admin para luego agregarla en la tabla de Ciudades.

-RF 2 - Registrar un Hotel



Solicita al Admin Nombre, Dirección, Teléfono, Descripción, Ciudad y el Admin para luego agregarlo a la tabla hoteles.

-RF 2.5 - Registrar un Servicio



Solicita al Admin: Nombre, Descripción, Tipo de Servicio, Precio, Id de la reserva, Número de la Habitación, Costo Total y Días de uso para luego agregarlo a la tabla de Servicios

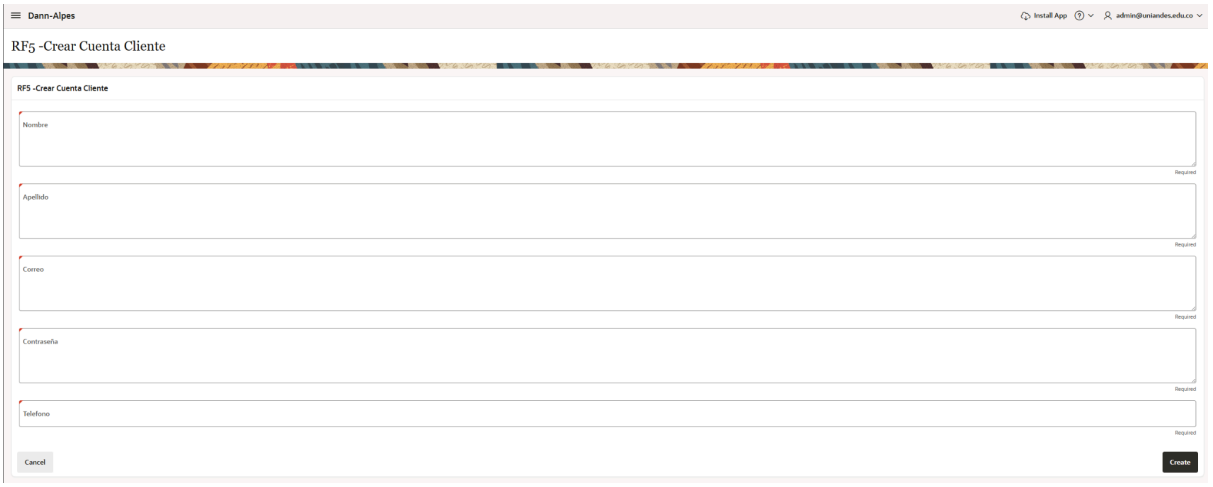
-RF 3 - Registrar un Tipo Habitación

Solicita al Admin: Nombre, Costo en T baja, Costo en T alta, Dimensiones de la habitación, Tipos de camas, Tipos de vistas, y si tiene o no los distintos tipos de comodidades para agregarla a la tabla TipoHabitacion.

-RF 4 - Editar una Habitación

Solicita al Admin el Nombre, los precios en temporada alta y baja, las dimensiones, tipo de cama, tipo de vistas, y las distintas comodidades para luego actualizar una habitación de la tabla Habitaciones.

-RF 5 Crear una cuenta de Cliente



Solicita Nombre, Apellido, Correo, Contraseña y Teléfono con valor predeterminado como cliente y luego lo agrega a la tabla Usuarios

-RF 6

-RF 7

5. Requerimientos Funcionales de Consulta

-RFC1

```sql
SELECT h.numero AS "Habitación",
       th.nombre AS "Tipo",
       th.costo_talta AS "Costo T. Alta",
       th.costo_tbaja AS "Costo T. Baja"
FROM Habitaciones h
JOIN TiposHabitaciones th ON h.tipo_habitacion = th.id
WHERE h.hotel = :P10_ID_HOTEL
AND h.numero NOT IN (
    SELECT hab.numero
    FROM Habitaciones hab
    JOIN Reservas r ON hab.id_reserva = r.id
    WHERE r.fecha_llegada < TO_DATE(:P10_FECHA_FIN, 'DD/MM/YYYY')
      AND r.fecha_salida > TO_DATE(:P10_FECHA_INICIO, 'DD/MM/YYYY')
)
```

Tablas usadas: Habitaciones (h), TiposHabitaciones(th), Reservas (r).

Joins: h.tipo_habitacion = th.id: Relaciona la habitacion fisica con su descripción y costos según el tipo.

Subconsulta: Se utiliza la relación entre Habitaciones y Reservas mediante hab.id_reserva = r.id.

Justificación: Se utilizó un INNER JOIN porque solo nos interesan las habitaciones que tienen un tipo asignado en el sistema. Para la exclusión de las habitaciones ocupadas se utilizó el NOT IN, asi el resultado solo muestra habitaciones que están libres en el rango de fechas solicitado.

-RFC2

```sql
SELECT TO_CHAR(r.fecha_llegada, 'DD/MM/YYYY') AS "Fecha de Entrada",
       COUNT(DISTINCT r.id) AS "Cantidad de Reservas"
FROM Reservas r
JOIN Habitaciones h ON h.id_reserva = r.id
WHERE h.hotel = :P11_ID_HOTEL
  AND r.fecha_llegada BETWEEN TO_DATE(:P11_FECHA_INI, 'DD/MM/YYYY')
                          AND TO_DATE(:P11_FECHA_FIN, 'DD/MM/YYYY')
GROUP BY r.fecha_llegada
ORDER BY r.fecha_llegada ASC;
```

Tablas usadas: Reservas (r), Habitaciones (h).

Joins: h.id_reserva = r.id: Permite filtrar las reservas que pertenecen.

Justificación: Se utilizó un INNER JOIN ya que una reserva debe estar vinculada obligatoriamente a una habitación para que se contabilice dentro de las estadísticas del hotel específico.

-RFC3

```
SELECT hot.nombre AS "Hotel",
       COUNT(r.id) AS "Total Reservas Activas",
       SUM(r.personas_adultas) AS "Adultos",
       SUM(NVL(r.menores_3_anios, 0)) AS "Niños < 3 años",
       SUM(r.personas_adultas + NVL(r.menores_3_anios, 0)) AS "Total Personas"
FROM Reservas r
JOIN Habitaciones h ON h.id_reserva = r.id
JOIN Hoteles hot ON h.hotel = hot.id
WHERE hot.id = :P12_ID_HOTEL
  AND TO_DATE(:P12_FECHA_CONSULTA, 'DD/MM/YYYY') BETWEEN r.fecha_llegada AND r.fecha_salida
GROUP BY hot.nombre
```

Tablas usadas: Reservas (r), Habitaciones (h), Hoteles (hot).

Joins: h.id_reserva = r.id: Vincula una reserva con la habitación ocupada.

h.hotel = hot.id: Vincula la habitación con el hotel correspondiente para obtener su nombre.

Justificación: Se utilizaron dos INNER JOIN, esto asegura que el reporte solo incluya datos de hoteles que tienen reservas en el momento de la consulta. Se utilizo NVL para el manejo de nulos en la columna de menores.

-RFC4

```
SELECT c.nombre AS "Ciudad",
       hot.nombre AS "Hotel",
       r.precio_total AS "Ingresos"
FROM Reservas r
JOIN Habitaciones h ON h.id_reserva = r.id
JOIN Hoteles hot ON h.hotel = hot.id
JOIN Ciudades c ON hot.ciudad = c.id
WHERE EXTRACT(YEAR FROM r.fecha_llegada) = :P13_ANIO;
```

Tablas usadas: Reservas (r), Habitaciones (h), Hoteles (hot), Ciudades (c).

Joins: h.id_reserva = r.id: Conecta el ingreso económico (reserva) con la habitación.

h.hotel = hot.id: Conecta la habitación con su hotel.

hot.ciudad = c.id: Conecta el hotel con la ciudad.

Justificación: Se utilizaron INNER JOIN, debido a que se requiere agrupar ingresos por ubicación, entonces se necesita la intersección entre las cuatro tablas.

-RFC5

```sql
SELECT th.nombre AS "Tipo de Habitación",
       COUNT(*) AS "Total Reservas"
FROM Reservas r
JOIN Habitaciones h ON h.id_reserva = r.id
JOIN TiposHabitaciones th ON h.tipo_habitacion = th.id
WHERE h.hotel = :P14_ID_HOTEL
  AND r.fecha_llegada BETWEEN TO_DATE(:P14_FECHA_INI, 'DD/MM/YYYY') AND TO_DATE(:P14_FECHA_FIN, 'DD/MM/YYYY')
GROUP BY th.nombre
ORDER BY "Total Reservas" DESC;
```

Tablas usadas (Consulta 1): Servicios (s), Habitaciones (h), Reservas (r).

Tablas usadas (Consulta 2): Reservas (r), Habitaciones (h), TiposHabitaciones(th).

Joins: s.numero_habitacion = h.numero y h.id_reserva = r.id: Para rastrear que servicios se pidieron durante una reserva especifica.

h.tipo_habitacion = th.id: para identificar que categorias de habitación son las mas buscadas

Justificación: Se utilizaron INNER JOIN, para poder medir la popularidad, solo nos interesan los servicios que sí fueron consumidos y las habitaciones que sí fueron reservadas. Los registros que no tienen esta vinculación no aportan al indicador de los más pedidos.

6. DESARROLLO DE LA APLICACIÓN EN APEX

-Cada requerimiento funcional y requerimiento funcional de consulta tuvo su respectiva página formulario en la app de apex

-Páginas implementadas:

7. ESCENARIOS DE PRUEBA

No realizamos ningún escenario de prueba